Railway Information Platform (RIP)

Project Design & Architecture Document

Version: 1.0
Author: Darshan Gaikwad
Tech Stack: FastAPI, Python, SQLite, HTML, CSS, JavaScript, Jira, GitHub

---

1. Project Overview

Objective

The Railway Information Platform is a backend-focused web application that provides railway-related services such as:

- PNR Status
- Live Train Status
- Train Between Stations
- Train Route
- Seat Availability
- Search History
- User Authentication (Future)
- AI Assistant (Future)

Initially, the project will use mock data stored in JSON files. As development progresses, the backend will be integrated with a live Railway/IRCTC-compatible API without changing the frontend.

---

2. Project Goals

Primary Goals

- Learn Backend Development using FastAPI
- Learn REST API Design
- Learn Software Architecture
- Learn Database Design
- Learn Git & GitHub Workflow
- Learn Agile using Jira
- Build a production-style portfolio project

---

3. Development Methodology

The project follows Agile Scrum.

Sprint Duration: 1 Week

Workflow:

Backlog → Sprint Planning → Development → Testing → Review → Deployment

Every feature will have:

- Epic
- Story
- Subtasks
- Story Points
- Acceptance Criteria

---

4. Current Roadmap

Sprint 1

- Project Setup
- FastAPI Setup
- PNR Validation API
- Mock Railway Service

Sprint 2

- Dynamic UI
- Search Result Page
- Error Handling

Sprint 3

- SQLite Integration
- Search History

Sprint 4

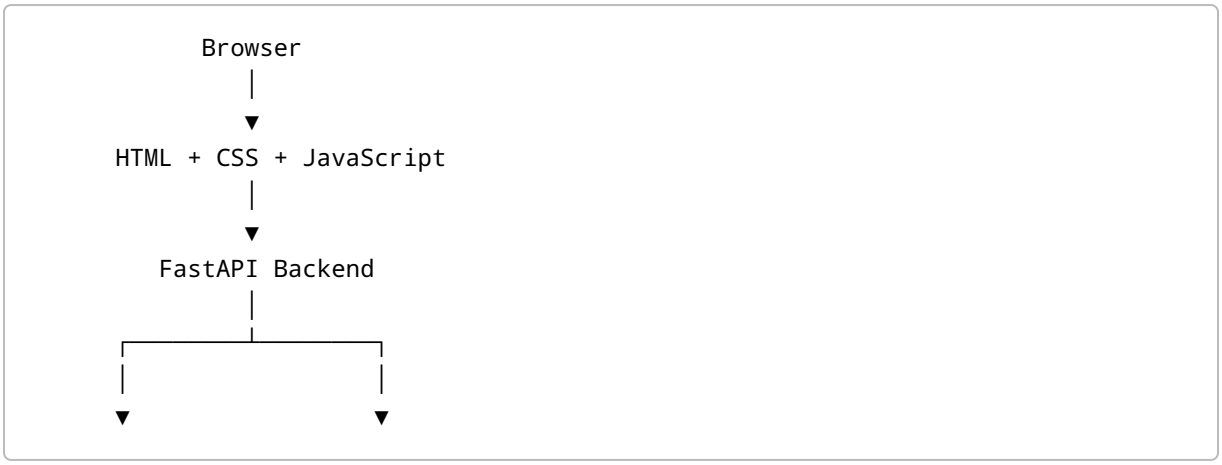
- Deployment
- Documentation
- GitHub Actions

Sprint 5

- Live Railway API Integration

---

5. High-Level Architecture



Business Services Validation | ▼ Data Source Layer | ┌──────┴──────┐ | | JSON Mock SQLite | | └──────┬──────┘ | ▼ External Railway API (Future Phase)

---

6. Layered Architecture

Presentation Layer

Responsible for:

- User Interface
- API Requests
- Displaying Data

Technology:

- HTML
- CSS
- JavaScript

API Layer

Responsible for:

- Receiving Requests
- Returning Responses
- Input Validation

Technology:

FastAPI

Business Layer

Responsible for:

- Business Logic
- Railway Rules
- Processing Requests

Examples:

check_pnr()

search_train()

get_live_status()

Data Layer

Responsible for:

- Reading JSON
- Database Operations
- External API Calls

Current:

Mock JSON

Future:

SQLite

Live Railway API

---

7. Folder Structure

railway-pnr-checker/

app/

```
main.py

routers/

services/

schemas/

models/

database/

data/

static/

templates/
```

tests/

requirements.txt

README.md

---

8. Request Flow

User enters PNR

↓

Frontend sends POST request

↓

FastAPI Router receives request

↓

Schema validates input

↓

Service searches data

↓

Returns matching train details

↓

Router returns JSON response

↓

Frontend displays information

---

9. Why We Use a Service Layer

Instead of placing all business logic inside the API route, we separate it.

Example

Bad Design

Route

↓

Reads JSON

↓

Loops through Data

↓

Returns Response

Good Design

Route

↓

Service

↓

JSON / Database / API

Benefits:

- Easier Testing
- Cleaner Code
- Reusable Functions
- Better Maintainability
- Easy to replace JSON with Live API

---

10. Current Data Source

Phase 1

JSON File

Advantages

- No Internet Required
- Easy Debugging
- Faster Development

Future

Railway API

No code changes required in the frontend because only the service layer changes.

---

11. Future Production Architecture

Browser

↓

Load Balancer

↓

FastAPI

↓

Redis Cache

↓

Business Services

↓

Railway API

↓

SQLite

↓

Logging

↓

Monitoring

---

12. Jira Workflow

Epic

↓

Story

↓

Subtasks

↓

In Progress

↓

Code Review

↓

Testing

↓

Done

---

13. Git Workflow

main

↓

feature/PNR-1-project-setup

↓

feature/PNR-2-fastapi

↓

feature/PNR-3-validation

↓

feature/PNR-4-service-layer

Each Story corresponds to one Git branch and one Jira issue.

---

14. Learning Outcomes

By completing this project, the following skills will be demonstrated:

- Python

- FastAPI
- REST APIs
- Pydantic
- SQLAlchemy
- SQLite
- JSON Handling
- API Integration
- HTML/CSS
- JavaScript Fetch API
- Git
- GitHub
- Jira
- Agile Scrum
- Software Architecture
- Deployment
- Docker (Future)
- CI/CD (Future)

---

15. End Goal

The final application will function as a Railway Information Platform capable of interacting with live railway services while following production-level software engineering practices.

The architecture has been intentionally designed so that replacing the mock JSON data source with a live Railway API requires changes only in the Service Layer, leaving the API routes and frontend unaffected.